

Teorema

mercoledì 30 marzo 2022 16:23

TEOREMA: SE L è un CFL allora esiste un PDA P tale che $L(P) = L$.

Gli automi PDA caratterizzano i linguaggi liberi dal contesto.

Il teorema è fatto da due parti:

1. SE L è un CFL esiste una CFG G tale che $L(G) = L$, allora esiste un PDA P tale che $L(P) = L = L(G)$

DA CFG $\rightarrow$ PDA

2. se ho un automa PDA, questo automa riconosce le stringhe di un linguaggio, che è in realtà context free (CFL). Da un automa posso costruire una stessa grammatica che riconosce questo linguaggio.

DA PDA $\rightarrow$ CFG

PROOF 1

• $G = (V, \Sigma, R, S)$

quello che vogliamo fare è costruire un automa PDA $P = (Q, \Sigma, \Gamma, \delta, q_0, F)$

Partendo da una S iniziale, io applico delle regole definite nella grammatica, per raggiungere una stringa terminale: ogni volta che voglio espandere posso scegliere un'arbitrarietà di regole sulla variabile da espandere e anche quale regola applicare poiché la stessa variabile può espandersi in stringhe differenti.

Questa arbitrarietà serve alla grammatica per poter generare tutte le stringhe del linguaggio:

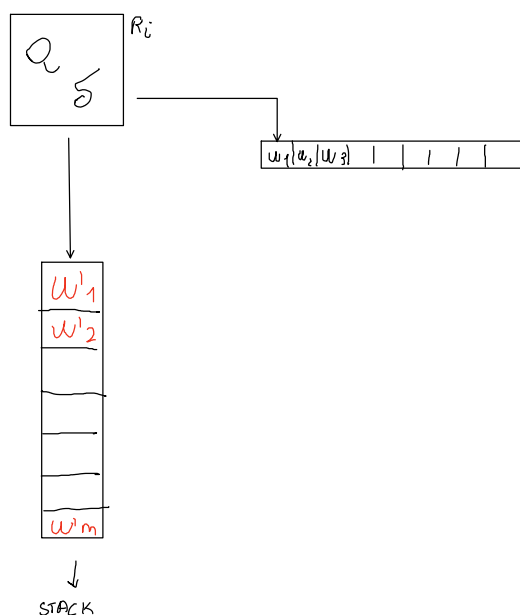
• Lo scopo è fornire un automa NON deterministico che faccia esattamente questo!

$S \rightarrow W \rightarrow^* \dots W$ $\rightarrow$ Perché io posso scegliere simultaneamente più regole!

L'automata dovrà esattamente fare le scelte giuste per poter riconoscere la stringa!!!

ORA IL NOSTRO AUTOMATA PRENDE IN INPUT W E DEVE RICONOSCERE SE APPARTIENE O NO AL LINGUAGGIO.

Trovare la macchina che riconosce tutte e sole le stringhe generate dalla grammatica!



$W = w_1 w_2 \dots w_m$

1. Costruiamo questa macchina in modo tale che non guardi l'input, ma simula la grammatica, usando il non determinismo.

CIOÈ parte da S e sostituisce tutto all'interno dello STACK sua ad associare ad una sequenza (dopo aver completato il ciclo di regole):

$w'_1 w'_2 \dots w'_m$

$\rightarrow$ questa è una delle infinite stringhe che la CFG può generare!

ho usato il non determinismo per simulare dove la grammatica sta leggendo l'input.

2. confronto se l'input che ho generato è uguale a quello sullo stack. se non lo è, ho preso una decisione sbagliata della regola. se c'è coincidenza, la stringa w è generata dalla grammatica, quindi accetto.

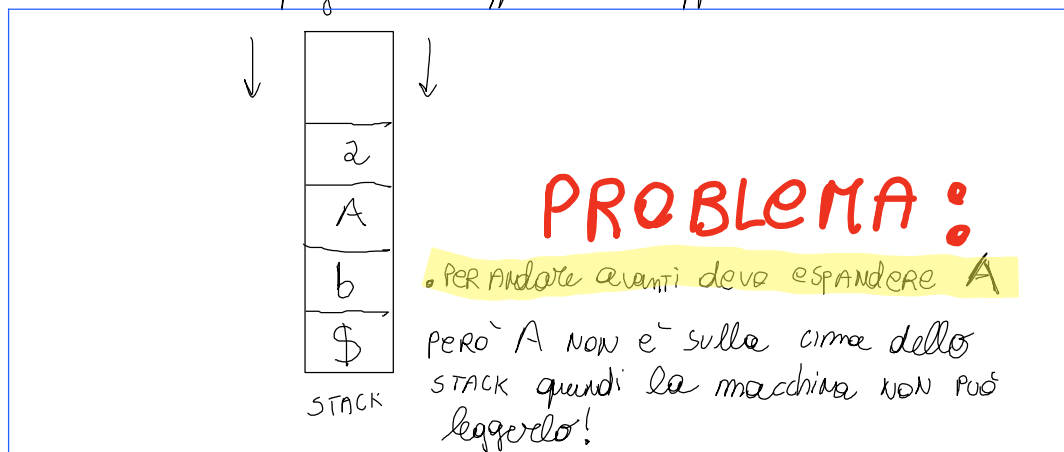
Perché quest'idea non funziona?

Supponiamo di avere una grammatica che ha questa regola:

$$S \rightarrow aAb$$

a e b sono terminali!

non deterministicamente indovino la regola da applicare, supponiamo a .

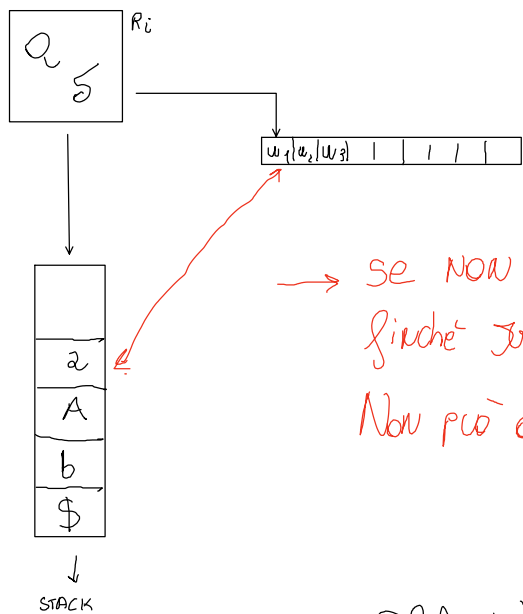


Per leggere A bisogna obbligatoriamente fare una pop di a . Però, se faccio la pop di a , me la scordo.

e se me la scordo come faccio a confrontare se poi quello che c'è nello stack è uguale all'input? È un problema.

Come lo RISOLVIAMO? Bisogna mischiare le due fasi.

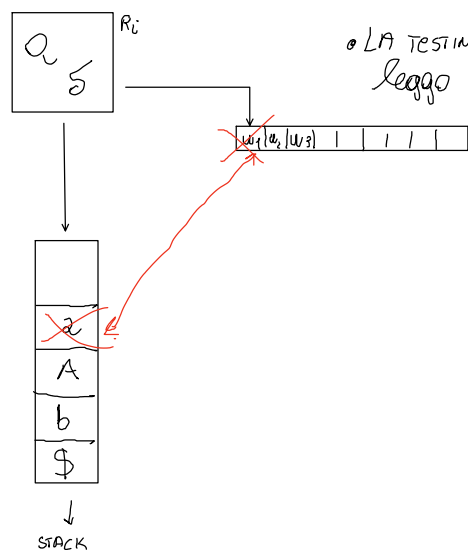
CONFRONTO SUBITO!



→ se non c'è corrispondenza, questa computazione RIFIUTA!
finché trovo terminali, procedo a confrontarli ed eliminarli.

Non può generare la stessa stringa, perché il primo carattere non è uguale.

ORA LI ELIMINO!



• LA TESTINA è andata avanti, nella cima dello stack ho una A non terminale, lo leggo e dico: "OK adesso lo espando".

↳ così al posto di A , metto a partire dal basso verso destra, l'espansione di A .

E procedo...

Questo funziona!

Vediamo come è fatto l'automa che riconosce la nostra grammatica.

$$Q = \{q_s, q_L, q_A\} \cup \{\text{stati intermedi per push stringhe}\}$$

← - ←

$\leftarrow \leftarrow \leftarrow \text{CFG}$

$$\Gamma = V \cup \Sigma \cup \{ \$ \}$$

$$F = \{ q_A \}$$

$$q_0 = q_s$$

$$\delta(q_s, \epsilon, \epsilon) = \{(q_L, S\$)\} \quad // \text{metto sullo stack } [S\$], \text{ da } q_s \text{ a } q_L$$

se sullo stack trovo A

↑



$$\delta(q_L, \epsilon, A) = \{(q_L, \epsilon, w) \mid A \rightarrow w\} \quad // \text{resti in } q_L, \text{ ma al posto di } A, \text{ metti la sua espansione.}$$

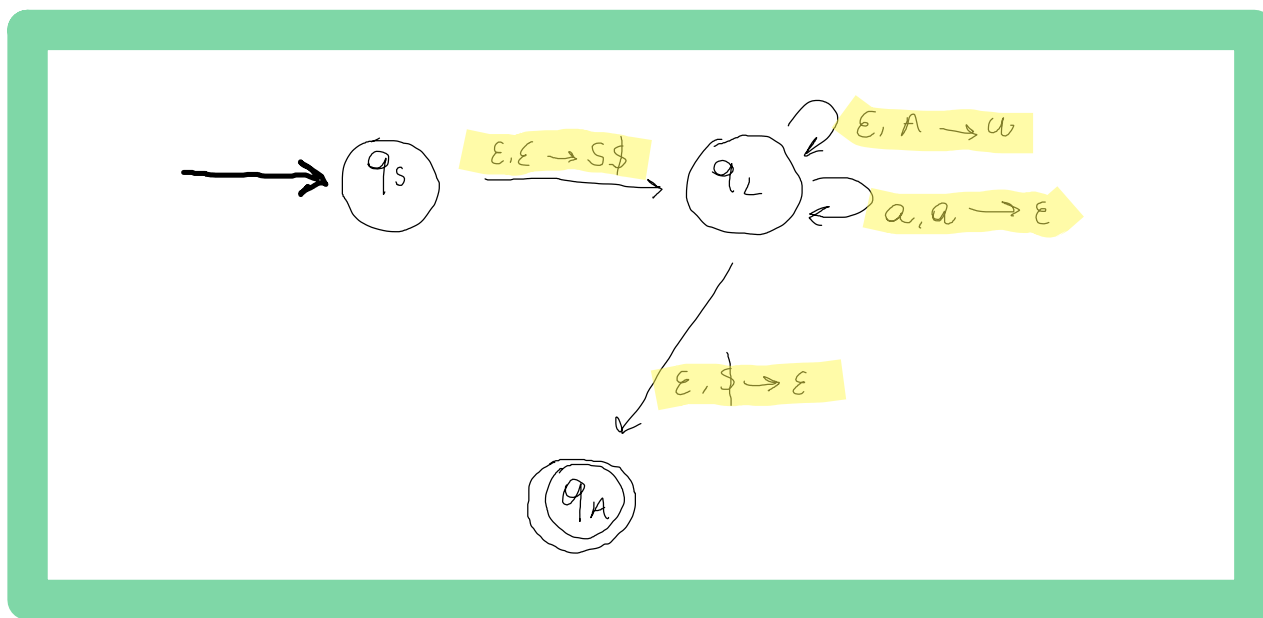
$$\delta(q_L, a, a) = \{(q_L, \epsilon)\}$$

se sullo stack trovo un terminale, tolgo dallo stack questa a: riconoscimento dei terminali in cima allo stack con la stringa di input.

$$\delta(q_L, \epsilon, \$) = \{(q_A, \epsilon)\} \quad // \text{da } q_L \text{ vado a } q_A, \text{ accetto quando sullo stack non mi rimane più nulla } (\$), \text{ perch  ho letto tutta la mia stringa di terminali.}$$

MAI MAI che ho letto la stringa, ho tolto dallo stack, il corrispondente simbolo.

$\forall \text{ regola } A \rightarrow w, \quad \forall a \text{ terminale}$



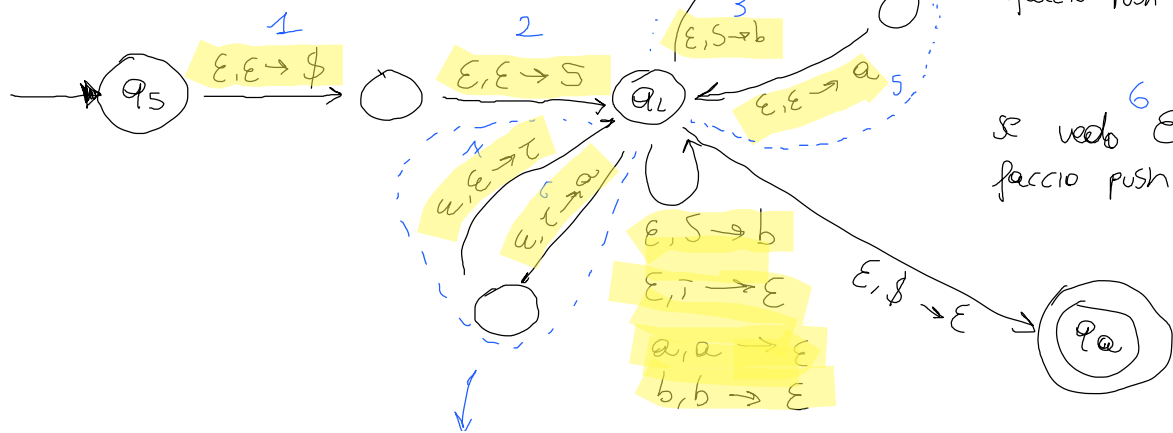
ESEMPIO:

DA CFG $\rightarrow$ PDA

Prendiamo una grammatica:

$$\begin{aligned} S &\rightarrow a T b \mid b \\ T &\rightarrow T a \mid \epsilon \end{aligned}$$

Vogliamo costruire un PDA!



Regola S

1,2 significare fare il push di dollaro ed S.
IN q_L devo avere una freccia per ogni regola!
Se vedo S, devo pushare ³ b, dopo pusho ⁴ T e faccio push di ⁵ a.

Se vedo ⁶ E e T faccio push di A e dopo faccio push di ⁷ T

PDA

Regola T

Accetta se e solo se lo stack è vuoto e l'input è finito!

PROOF 2

capiamo come è fatta la macchina.

La nostra macchina, in generale, è fatta così:

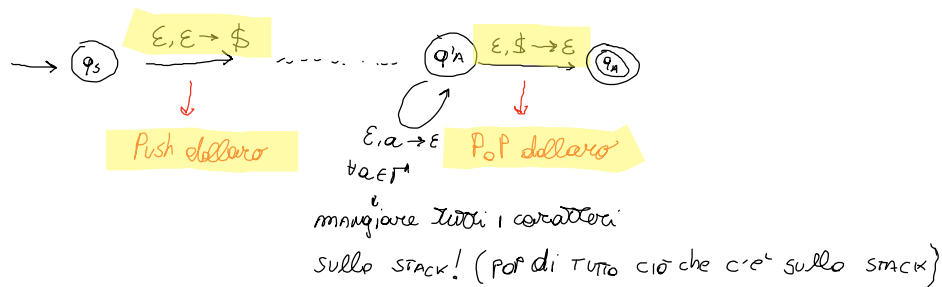
$$P = (Q, \Sigma, \Gamma, \delta, q_0, F).$$

Moniamo delle Regole:

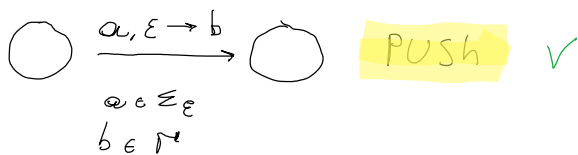
1) $|F| = 1$, c'è solo uno stato di accettazione!

2) P accetta lasciando lo stack vuoto!

L'INPUT è consumato e mi ritrovo in uno stato di accettazione. In questo stato di accettazione, la macchina, possa svuotare tutto quello che c'è sullo stack.

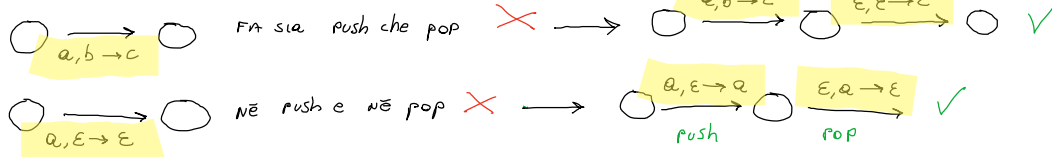


3) Qualunque transizione che fa la macchina c', o una PUSH, o una POP, NON ENTRAMBE!



queste due sono ammesse

queste non sono ammesse:



A questo punto, posso descrivere una grammatica che simuli questa macchina con RESTRIZIONI.

$$P = (Q, \Sigma, \Gamma, \delta, q_0, \{q_A\})$$

↳ unico stato di accettazione q_A .

DOBBIAMO COSTRUIRE UNA GRAMMATICHE: (il linguaggio riconosciuto da P deve essere uguale al linguaggio generato da G)

$$G = (V, \Sigma, R, S)$$

Come è fatta questa grammatica?

Le variabili di V le possiamo vedere come "indicate" da P e Q , dove P e Q sono stati INTERNI della macchina:

$$V = \{ A_{pq} \mid p, q \in Q \}$$

Per ogni coppia di stati interni p e q della macchina, c'è una variabile!

Ora possiamo anche definire lo stato iniziale della macchina:

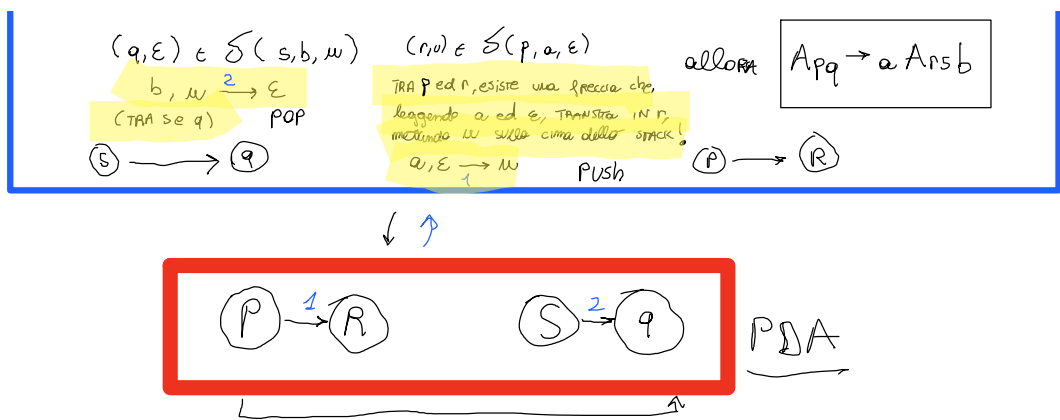
$$S = A_{q_0 q_A}$$

R CONTIENE Regole:

$$\forall p \in Q, A_{pp} \rightarrow \epsilon$$

$$\forall p, q, r \in Q, A_{pq} \rightarrow A_{pr} A_{rq}$$

$$\forall p, q, r, s \in Q, \forall a, b \in \Sigma_\epsilon, \forall u \in \Gamma$$



$A p q \rightarrow^*$ da qui si può generare una sequenza di terminali che permettono di portare da P a Q
 Con il vincolo di lasciare lo stack vuoto all'inizio e alla fine!

Bisogna cominciare di questo fatto. la dimostrazione verrà effettuata in due parti.

- se c'è una stringa che porta la macchina da P a Q allora $A p q$ riesce a generare quella stringa
- se $A p q$ genera una stringa, quella porta la variabile da p a q .

Una volta che dimostro queste due, ho finito! Perché la nostra variabile di partenza è $A q_0 q_A$, e questa variabile genera tutte le stringhe che possono portare dallo stato q_0 a q_A .
 Dallo stato iniziale allo stato finale.

Se dimostro questo, il linguaggio generato da questa grammatica è esattamente il linguaggio delle stringhe riconosciute dall'automa: $DA PDA \rightarrow CFL$.

Cominciamo a dimostrare queste due cose: Partiamo dalla seconda!

Se, $A p q \rightarrow^* x$, $A p q$ genera una stringa x , allora x porta P da P a Q , con lo stack vuoto

con lo stack vuoto, non significa che lo stack rimanga sempre vuoto, ma che all'inizio è vuoto e alla fine è vuoto.

Dimostrazione:

Devo farlo per tutte le stringhe x e quindi lo faccio "per induzione".
 per induzione sulla lunghezza di x .

Base: lunghezza di $x=1$: $A p q \rightarrow x$!

x è una stringa terminale. Qual'è l'unica regola applicabile in cui a destra ci sono soltanto terminali?

questo mi fa dedurre che x è la stringa vuota e che $P=Q$.

Quindi la stringa vuota ϵ , porta la macchina da uno stato interno P (con lo stack vuoto) allo stato interno P (con lo stack vuoto).

MA se la stringa è vuota, la macchina non fa nessun passo, è ovvio che è vero! ✓
Rimango nello stato p e lo stack non si muove.

Passo induttivo: lunghezza di $|y| \leq k$. $A_{rs} \rightarrow^* y$. (supponiamo vada bene)

y^* porta la macchina dallo stato r (con lo stack vuoto) allo stato s (con lo stack vuoto).

Poniamoci il problema di una certa situazione:

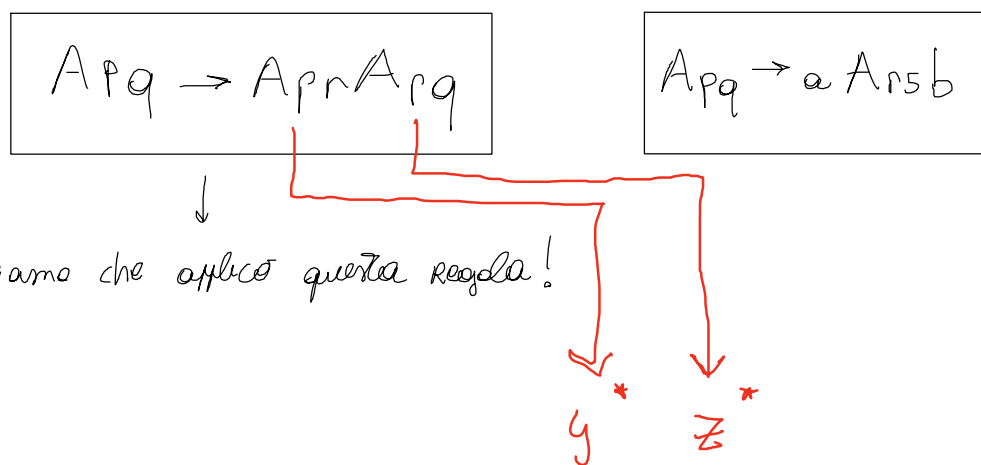
$A_{pq} \rightarrow^* x$, dove la lunghezza di $x \leq k+1$

Come può essere fatta questa derivazione? Una derivazione di $k+1$ passi, comincia sempre con un primo passo.

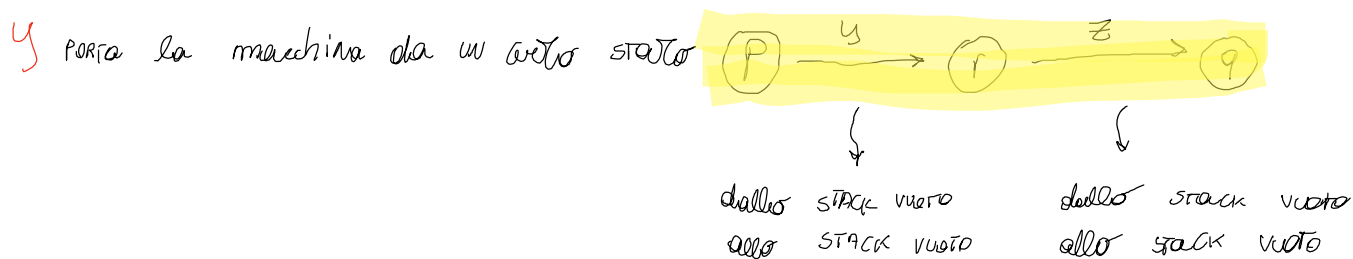
Questo primo passo è l'applicazione di una regola, e questa regola la devo scegliere tra quelle che avevamo imposte.

La prima regola non è, perché io la posso applicare solo se $p=q$.

Quindi possiamo applicare solo le altre due!



Ma la lunghezza di queste derivazioni sarà uguale al più a k , sommate le due derivazioni daranno lunghezza k : ciascuna di loro sarà più piccola o uguale a k .



Questo significa che, se io concateno yz , questa stringa porterà la macchina P dallo stato p con lo stack vuoto, allo stato q con lo stack vuoto!

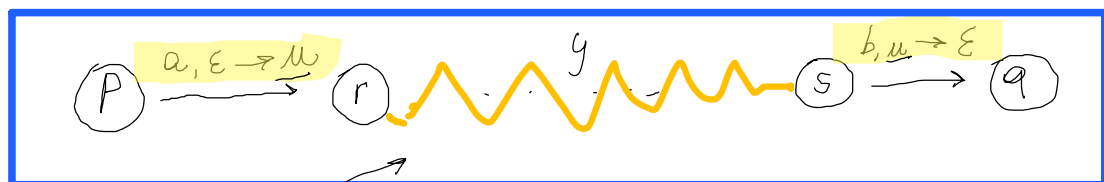
Nello stato r , lo stack si riavvera, cioè ritorna a vuoto.

Quindi rimane confermato che $A_{pq} \rightarrow^* x$ va da p a q con stack vuoto!

(In mezzo lo stack può crescere, ma alla fine torna vuoto: da dimostrare)

Ora supponiamo di applicare $A_{pq} \rightarrow a A_{rs} b$ **espande in y (stringa di terminali) con $\text{len} \leq k$**

Purché questa regola si possa applicare deve esistere

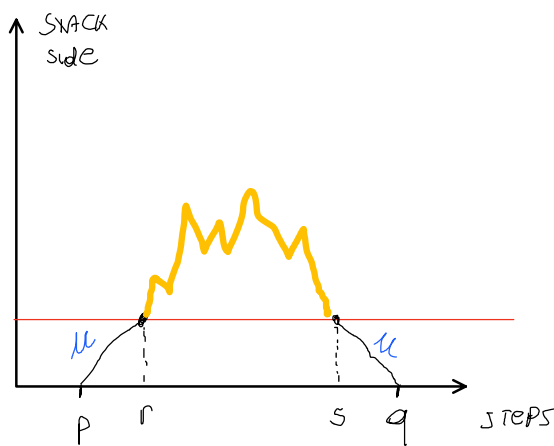


→ **mi porta la macchina P dallo stato r allo stato s , consumando la stringa y , dallo stack vuoto allo stack vuoto.**

Ma lo stack qui non è veramente vuoto, perché subito prima io ho fatto la push di una u , e dopo la pop di u .

Tra r ed s non si va mai al di sotto!

Quindi in p e q lo stack è vuoto, quindi la stringa $x = ayb$ porta la macchina dallo stato p allo stato q con lo stack vuoto.



quindi abbiamo dimostrato il problema di questa situazione! ↑

Prima dimostrazione fatta ✓

ora dimostriamo:

- se c'è una stringa che porta la macchina da P a q allora A_{pq} riesce a generare quella stringa

dimostrazione

Supponiamo che esista una stringa di terminali X che:

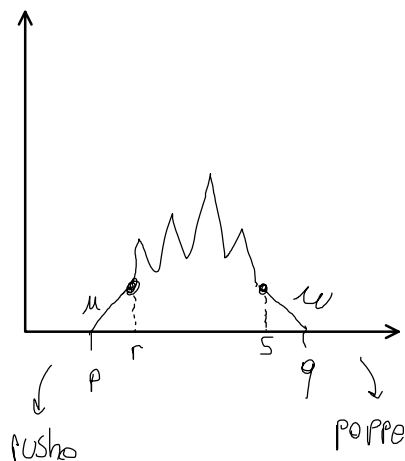
$(P) \xrightarrow{X} (q)$ che porta da P a q lasciando lo stack vuoto.

DOBBIAMO dimostrare che $A_{pq} \rightarrow^* X$ (A_{pq} può generare X).

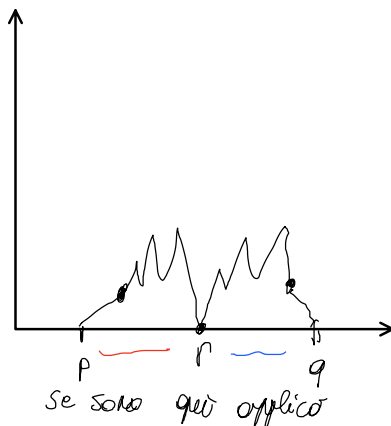
Dimostriamolo per induzione:

Base: lunghezza 0, se ha 0 PASSI la macchina passa da p a q , leggendo niente: questo si può fare perché esiste una regola $A_{PP} \rightarrow \epsilon$!

Passo induttivo:



$A_{pq} \rightarrow a A_{rs} b$
come prima! $\rightarrow a y b \rightarrow^* X$



se sono qui applico

$A_{pq} \rightarrow A_{pr} A_{rq}$

e ora applico l'ipotesi induttiva.
finito.

Quindi, SE ho un PDA, posso costruire una CFG equivalente

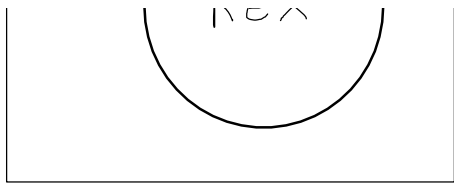
COROLLARIO: i linguaggi regolari sono CFL.

Se un linguaggio è regolare, esiste un NFA che lo riconosce!

MA, se esiste un NFA, esiste un PDA: qualunque NFA è un PDA che non usa mai lo stack.

Se esiste un PDA è CFL.





se ho un linguaggio che non è CFL, come faccio a dimostrare che non è CFL?

devo dimostrare la "non regolarità" del linguaggio:

PUMPING LEMMA →